Impact of Activation Functions on Gradient Propagation and Training Stability in Deep Neural Networks

Name : Sujith Reddy Punreddy **Student id:** 24151457

Github Repository : https://github.com/sujithpunreddy/activation-functions-vanishing-gradient

1. Introduction to Neural Networks

Artificial neural networks are widely used in modern machine learning for tasks such as image recognition, speech processing and recommendation systems. Their main strength is the ability to learn useful patterns directly from data instead of relying on manually designed rules.Neural networks are inspired by biological neurons in the human brain. Artificial neurons receive input values, multiply them by adjustable weights and produce outputs after applying a transformation function. When many neurons are arranged in layers, they form deep neural networks that can learn hierarchical representations and solve complex problems.The objective of this tutorial is to explain how activation functions influence gradient propagation, convergence speed and classification performance in deep neural networks.

2. Importance of Activation Functions in Neural Networks

Activation functions are important in neural networks because they introduce non-linearity into the learning process. Many real-world problems cannot be solved using simple linear relationships. For example, recognising objects in images requires understanding complex patterns such as shapes and textures. If a neural network only performs linear operations, adding more layers will not increase its learning ability because multiple linear transformations can be combined into a single linear function.By applying an activation function after each weighted sum, neural networks can learn complex decision boundaries and extract meaningful features from data. This makes deep learning systems more effective than traditional linear models.Several activation functions are used in practice. The sigmoid function maps input values into a range between zero and one and is useful for probability-based predictions. However, sigmoid neurons may enter saturated regions where gradient values become very small, slowing down learning. The hyperbolic tangent function produces outputs between negative one and positive one and can provide slightly more stable optimisation because its outputs are centred around zero.In modern deep learning, the Rectified Linear Unit (ReLU) is widely used because of its simple computation and ability to maintain stronger gradient signals. This often leads to faster convergence in deep neural networks.

3. Learning Process in Neural Networks

Neural networks learn by adjusting internal parameters so that predictions become closer to the correct outputs. This process is guided by a loss function that measures prediction error. During training, the network first performs a forward pass where input data moves through each layer to generate outputs.After computing the loss, back propagation is used to determine how each parameter contributed to the error. This method applies the chain rule of calculus to calculate gradients of the loss with respect to each weight. These gradients indicate how weights should be updated to reduce error.An optimisation algorithm such as gradient descent or Adam is then applied

to update parameters step by step. Over multiple iterations, the network gradually learns useful patterns from the data. The success of this process depends on how effectively gradient information flows through the layers.

4. The Vanishing Gradient Problem

Training deep neural networks can be difficult because gradients must travel backwards through many layers. At each layer, gradients are multiplied by the derivative of the activation function. When these derivatives are small, gradient values shrink as they move towards earlier layers.This issue is known as the vanishing gradient problem. When gradients become extremely small, early layers receive limited learning signals and weight updates become insignificant. As a result, the network may struggle to learn meaningful features and training becomes slow.Activation functions such as sigmoid and tanh are more likely to cause vanishing gradients because their derivatives are bounded between zero and one. When neurons operate in saturated regions, gradient values approach zero. In contrast, ReLU helps maintain stronger gradients for positive inputs, making deep networks easier to train.

5. Mathematical Insight into Gradient Behaviour

To understand the vanishing gradient problem, it is useful to examine how gradients are calculated during training. Neural networks use gradient-based optimisation methods where parameters are updated using derivatives of the loss function. These derivatives are computed using the chain rule of calculus.In deep networks, gradients for earlier layers are obtained by multiplying several derivative terms. If the average derivative value is less than one, repeated multiplication causes gradient magnitude to decrease rapidly. For example, if each derivative is around 0.5, the final gradient becomes very small after passing through multiple layers. This limits learning in early layers.Another factor influencing gradient behaviour is weight initialisation. Poor initialisation may cause activations to saturate before training begins. Modern approaches such as He initialisation are designed for ReLU networks and help maintain stable signal variance across layers.During backpropagation, the gradient of the loss with respect to early layer weights is computed using repeated application of the chain rule. This results in multiplication of many derivative terms. When these values are smaller than one, gradient magnitude decreases exponentially across layers, making optimisation difficult and slowing convergence.

6. Experimental Study Using the Fashion-MNIST Dataset

To understand the practical impact of activation functions, an experimental study can be performed using the Fashion-MNIST dataset. This dataset contains grayscale images of clothing items such as shirts, shoes, coats and bags. Each image has a size of 28 × 28 pixels, which makes the dataset suitable for training neural networks without requiring very high computational power.Before training the model, several pre-processing steps are necessary. Pixel values are usually normalised so that they fall between zero and one. This helps improve numerical stability during optimisation and allows gradients to propagate more smoothly. The dataset is also divided into training and validation sets so that model performance can be evaluated on unseen data.To fairly compare activation functions, the same neural network architecture is trained multiple times while changing

only the activation function. Other parameters such as learning rate, batch size, optimiser and number of epochs are kept constant. This controlled setup ensures that any differences in performance are mainly due to the behaviour of activation functions rather than unrelated factors.

7. Neural Network Architecture and Training Behaviour

A deep fully connected neural network can be used to perform the image classification task. In this architecture, the input layer first converts each image into a one-dimensional vector so that it can be processed numerically. Several hidden layers are then used to transform the input features into more useful internal representations. Each hidden layer performs a linear transformation followed by an activation function.Increasing the number of hidden layers allows the network to learn more complex patterns from the data. However, deeper architectures also make the optimisation process more difficult because gradients must pass through a larger number of transformations. Weak gradient signals can prevent early layers from learning basic features.During training, learning behaviour can be analysed by observing how the loss value changes across epochs. Networks using sigmoid activation often show slower improvement in loss because gradient signals become weak. Models using tanh activation may perform slightly better due to more balanced gradient flow. In contrast, networks using ReLU typically show faster reduction in loss and achieve higher accuracy within fewer training iterations.These differences demonstrate that activation function selection has a direct impact on training efficiency and model performance.

8. Interpretation of Experimental Results

The comparison of different activation functions shows clear differences in how neural networks learn. When sigmoid activation is used in deep networks, the reduction in training loss is usually slow and gradual. This suggests that gradient signals are weak and weight updates are small. As a result, the model may require more training time and may not achieve the highest possible accuracy.

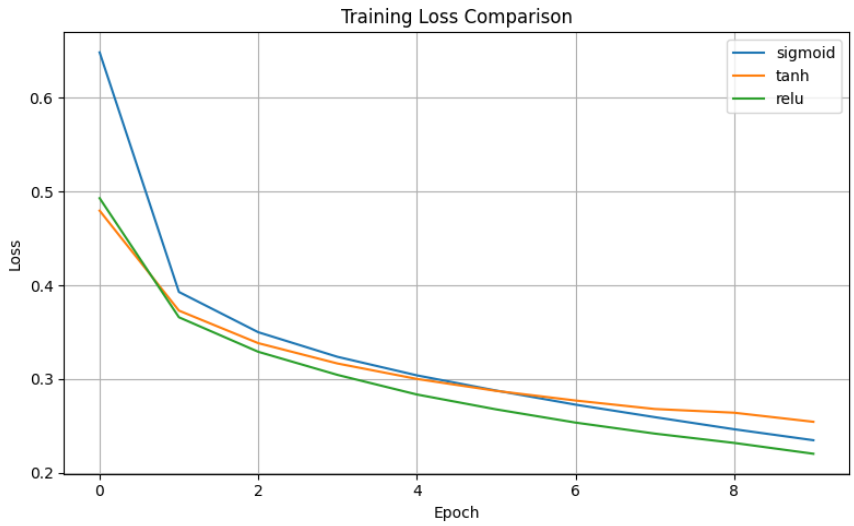

Figure 1: Training loss comparison for sigmoid, tanh and ReLU activation functions.

In contrast, networks that use ReLU activation generally demonstrate faster convergence. Training loss decreases more rapidly during early epochs, indicating that gradients are propagating more effectively through the network. Validation accuracy also tends to increase more quickly. Visualising loss curves and accuracy plots helps learners understand how activation functions influence optimisation behaviour and model efficiency.These observations highlight the practical importance of selecting suitable activation functions when designing deep learning systems.

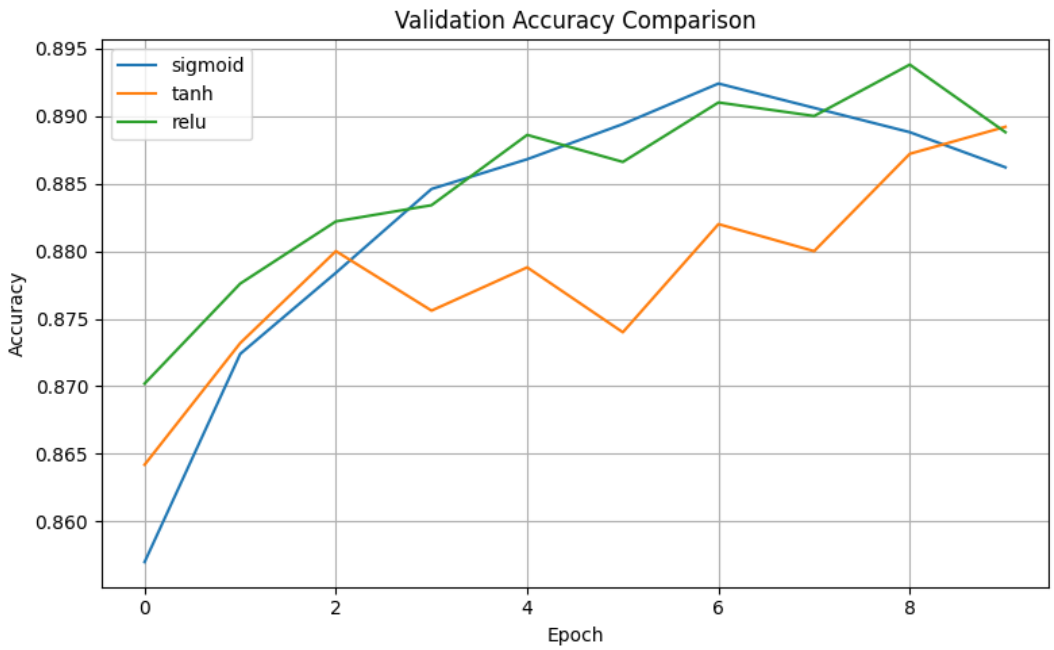

Figure 2: Validation accuracy comparison for sigmoid, tanh and ReLU activation functions.

9.Limitations of Activation Functions and Modern Improvements

Although activation functions such as ReLU have improved the training of deep neural networks, they are not without limitations. One common issue occurs when neurons receive negative inputs for long periods during training. In such cases, the output of a ReLU neuron becomes zero, and the gradient flowing through that neuron may also become zero. This situation can reduce the learning capacity of the network because some neurons stop updating their weights.To address this problem, researchers have proposed several modified activation functions. One example is Leaky ReLU, which allows a small positive gradient even when the input is negative. Another variation is Parametric ReLU, where the slope of the negative region can be learned during training. These approaches aim to maintain the benefits of ReLU while improving gradient flow in different learning situations.Modern deep learning architectures also use additional techniques to stabilise training. Batch normalisation helps maintain consistent activation distributions across layers, which can improve convergence speed. Residual connections allow gradient information to bypass certain layers and reach earlier stages more effectively. These innovations have enabled the successful training of very deep neural networks used in many real-world applications.

10. Real-World Importance of Activation Function Selection

Choosing an appropriate activation function is not only a theoretical concern but also an important practical decision in machine learning applications. In real-world systems such as medical diagnosis, fraud detection and autonomous driving, neural networks must learn reliable patterns from complex datasets. If gradient signals become too weak during training, the model may fail to extract meaningful features, which can reduce prediction accuracy and system reliability.

11 . Experimental Limitations and Future Scope

Although the experiment provides useful insights, it is limited to a fully connected architecture and a relatively shallow network depth. More complex architectures such as convolutional neural networks or very deep residual networks may show different gradient behaviours. Future work could explore these aspects and analyse activation functions in larger-scale deep learning systems.

12. Conclusion

Activation functions play an essential role in neural networks because they allow models to learn complex and non-linear patterns from data. Without activation functions, deep neural networks would behave like simple linear models and would not be able to solve difficult tasks such as image recognition or speech processing. This tutorial explained how different activation functions influence the learning process and why their selection is important for effective model training.A major challenge discussed in this work is the vanishing gradient problem, which occurs when gradient signals become very small as they move backwards through many layers. This can slow down learning and reduce model performance. Activation functions such as sigmoid and tanh are more likely to cause this issue, while modern functions like ReLU help maintain stronger gradient flow and support faster convergence.Overall, understanding activation behaviour, optimisation methods and network design principles is important for building reliable machine learning systems. By combining theoretical knowledge with practical experimentation, neural networks can be trained more efficiently and applied successfully in real-world applications.

Accessibility Considerations :

This tutorial is written using clear language and structured headings to improve readability. All plots use colour-blind friendly palettes and include descriptive captions. The code is organised to support reproducibility and ease of understanding for new learners.

References

- *Goodfellow, I., Bengio, Y. and Courville, A. (2016) Deep Learning. Cambridge, MA: MIT Press.*
- *LeCun, Y., Bengio, Y. and Hinton, G. (2015) Deep learning. Nature, 521(7553), pp. 436–444.*
- *Chollet, F. (2021) Deep Learning with Python. 2nd edn. Manning Publications.*
- *Nair, V. and Hinton, G. (2010) Rectified linear units improve restricted Boltzmann machines. In: Proceedings of the International Conference on Machine Learning.*
- *He, K., Zhang, X., Ren, S. and Sun, J. (2016) Deep residual learning for image recognition. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition.*
- *Ioffe, S. and Szegedy, C. (2015) Batch normalization: Accelerating deep network training by reducing internal covariate shift. In: Proceedings of the International Conference on Machine Learning.*